

Normalisation du processus de développement et plan de mise en œuvre de la chaîne CI

Le plan présenté dans ce document est proposé à titre d'exemple, illustrant la structure attendue dans le cadre d'un projet réel.

Dans le contexte de cet exercice, ne cherchez pas à détailler chaque section de manière excessive, cela prendrait trop de temps.

Considérez ce document comme un guide plutôt qu'une liste d'attentes strictes à respecter à la lettre.

1. Introduction.....	2
1.1. Objectifs.....	2
1.2. Contexte.....	2
9. Normes et bonnes pratiques.....	3
9.1. Principes de base.....	3
9.2. Conventions d'écriture de code et règles de style.....	3
9.3. Gestion des versions.....	3
9.4. Revue de code.....	3
9.5. Intégration continue.....	3
9.6. Tests.....	3
9.7. Sécurité.....	3
9.8. Documentation.....	3
10. Plan de mise en œuvre.....	3
10.1. Objectifs.....	3
10.2. Étapes.....	4
10.2.1. <Etape X>.....	4
10.3. Ressources nécessaires.....	4
10.4. Calendrier prévisionnel.....	4
10.5. Gestion des risques.....	4
11. Conclusion.....	4
11.1. Synthèse.....	4
11.2. Prochaines étapes.....	4

1. Introduction

1.1. Objectifs

Ce document a pour objectif d'établir un point de départ vérifiable, puis de formaliser les normes et le plan de mise en œuvre de la future chaîne CI de MicroCRM. À l'étape D.8, seule la baseline issue de l'audit est reportée ; les pratiques cibles, les outils, les responsabilités, les ressources et le calendrier seront finalisés après la veille technologique.

1.2. Contexte

MicroCRM est un monorepo GitLab composé d'un frontend Angular et d'un backend Spring Boot. La configuration versionnée contient une intégration continue minimale organisée en deux stages, test et build, avec quatre jobs séparés pour les deux composants.

2. Baseline fonctionnelle — Les mesures locales du 26 juillet 2026 ont confirmé la réussite de 8 tests frontend et de 2 tests backend, ainsi que des builds Angular et Gradle. Les trois cibles Docker existantes ont également pu être construites.
3. Limites de la CI — La chaîne s'arrête aux tests et aux builds. Elle ne conserve ni rapport, ni couverture, ni artefact, ni image publiée. Certaines opérations sont répétées, notamment l'installation npm et l'exécution des tests backend pendant le build.
4. Qualité et sécurité — Les tests actuels couvrent peu les comportements métier : aucun scénario HTTP ou CRUD complet n'est présent. La couverture frontend mesurée atteint 30,76 % des lignes et 9,52 % des branches. Aucun contrôle continu de qualité, de dépendances, de secrets ou d'images n'est intégré ; la baseline npm signale 64 vulnérabilités, dont une critique et 46 élevées.
5. Livraison et traçabilité — Les images sont construites, contrôlées et transmises manuellement selon les équipes. Aucun identifiant immuable ou manifeste ne relie encore une image au commit, aux tests et aux contrôles qui l'ont produite.
6. Portabilité et données — L'URL d'API localhost est intégrée au frontend, le port déclaré par l'image backend ne correspond pas au port Spring Boot, la configuration Caddy n'a pas permis le smoke test attendu et aucun healthcheck n'est défini. La base HSQLDB actuelle perd les données au remplacement du conteneur et la configuration CORS reste permissive.
7. Périmètre de la baseline — Ces observations proviennent du dépôt, des sondages Dev/Ops et de mesures locales effectuées sur la branche dev au commit 526bd96cddd2903676988b56dfb2778667aa435. Aucun pipeline GitLab distant, test de charge, environnement AWS ou indicateur de production n'a été observé. Le détail des preuves reste centralisé dans le document de veille, d'audit et de recommandations.
8. Suite du document — La cible de normalisation ne sera pas déduite de cette seule baseline. Les options seront comparées pendant la veille, puis les normes, étapes, responsables, ressources, risques et dates seront arrêtés dans la phase de conception.

9. Normes et bonnes pratiques

9.1. Principes de base

Énoncer les principes fondamentaux qui guident la normalisation du processus de développement (par ex: automatisation, répétabilité, sécurité, qualité, etc).

9.2. Conventions d'écriture de code et règles de style

Décrire les normes de codage qui doivent être suivies par les développeurs (par ex: convention de nommage, organisation du code, documentation, etc).

9.3. Gestion des versions

Définir les règles pour la gestion des versions, y compris l'utilisation des branches (par ex: gitflow, trunk-based, fork and pull, etc.)

9.4. Revue de code

Établir les procédures pour la revue de code, y compris les critères d'acceptation et les outils utilisés (par ex: pull requests, mail patch, etc).

9.5. Intégration continue

Spécifier les pratiques d'intégration continue, y compris la fréquence des intégrations, les critères de réussite et les outils CI (par ex: GitLab CI).

9.6. Tests

Définir les tests à automatiser dans la chaîne CI, y compris la fréquence d'implémentation.

9.7. Sécurité

Définir les pratiques de sécurité appliquées dans le processus de développement (par ex: scans de sécurité, gestion des secrets, audit de sécurité).

9.8. Documentation

Définir les exigences en matière de documentation à chaque étape du processus de développement (ex: documentation du code, documentation utilisateur).

10. Plan de mise en œuvre

10.1. Objectifs

Rappeler les objectifs du plan de mise en œuvre et les résultats attendus.

10.2. Étapes

Lister les étapes de mise en œuvre en mettant l'accent sur l'objectif de chaque étape et les livrables qui devraient être associés à la réalisation de chaque étape.

10.2.1. <Etape X>

- **Description** : Décrire l'objectif de réalisation lié à l'étape et les actions concrètes à mener
- **Responsables** : Identifier les responsables en charge de la réalisation
- **Livrables** : Identifier les livrables, c'est à dire les productions concrètes liées à la finalisation de l'étape (par ex: rapport d'audit, documentation, pipeline CI configuré et fonctionnel, etc)

10.3. Ressources nécessaires

Lister les ressources nécessaires pour la mise en œuvre (par ex: personnel, budget, environnements techniques, etc).

10.4. Calendrier prévisionnel

Proposer un calendrier prévisionnel de réalisation des étapes précédemment listées, dans un cadre normal de fonctionnement (toutes les ressources disponibles, sans difficulté ou écueil important rencontré).

10.5. Gestion des risques

Identifier les risques potentiels liés à la mise en œuvre et les stratégies pour les atténuer s'ils venaient à se présenter.

11. Conclusion

11.1. Synthèse

Synthétiser les principaux objectifs de la normalisation du processus de développement et les avantages apportés dans le cadre du projet.

11.2. Prochaines étapes

Identifier les prochaines étapes après la mise en œuvre pour assurer le succès à long terme de cette stratégie de normalisation.